Flappy Bird Game

p5play JS library

Lesson 7 Display Score

4 Sept 2025

Learning Objectives

- Note each digit image size, what is the width and height?
- Lots of concepts for this topic
    - Arrays to manage a list of digit (0-9) images
    - Loop through an Array
    - Conversion from number to String, to Array
    - (p5play) Sprite Group, and inheritance
    - Adjustment to place digit one after another (calculation)
    - Adjustment to ensure all digits are displayed in the centre (but camera is scrolling)

Step 1: Use JS Array instead of using 10 variables, to load 10 digits

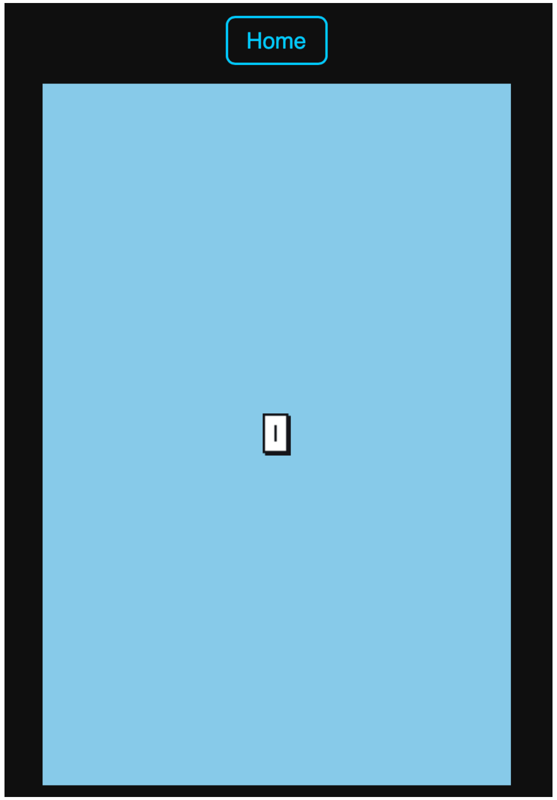
```
let digitImgs = [];
let digit0;

let scoreNumber = 0;
let scoreSprite;

```

> In the preload() function, load a single digit

```
    digit0 = loadImage('assets/0.png');
```

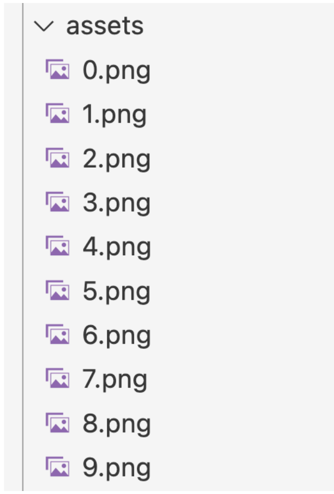
> Use a sprite to load the 0 digit image

```
function setup() {
    new Canvas(400,600);

    scoreSprite = new Sprite();
    scoreSprite.img = digit0;
    scoreSprite.collider = "none"; // explain static, dynamic, none
}

function draw() {
    background("skyblue"); // erase the trail left behind
}

```



Step 2: Use 10 variables, to load 10 digits

```
function preload() {
    digit0 = loadImage('assets/0.png');
    digit1 = loadImage('assets/1.png');
    digit2 = loadImage('assets/2.png');
    digit3 = loadImage('assets/3.png');
    digit4 = loadImage('assets/4.png');
    digit5 = loadImage('assets/5.png');
    digit6 = loadImage('assets/6.png');
    digit7 = loadImage('assets/7.png');
     digit8 = loadImage('assets/8.png');
     digit9 = loadImage('assets/9.png');
```

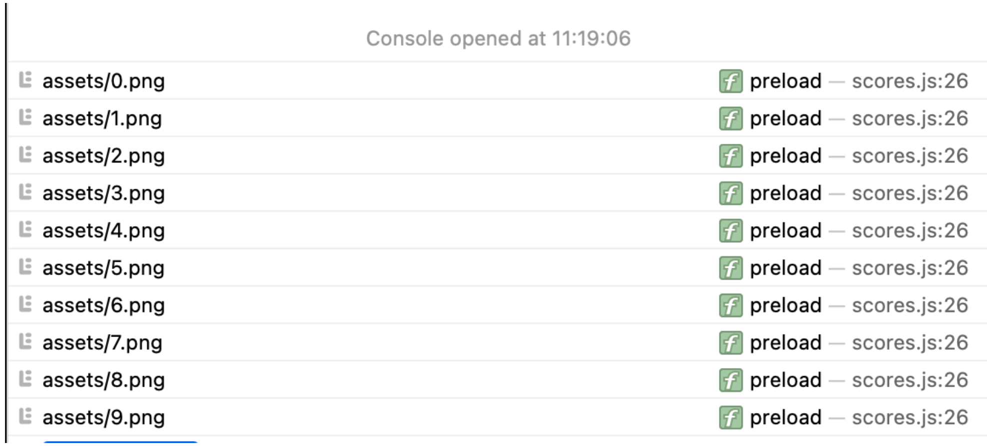


***Take note of the filenames.**

Step 2: Use 10 variables, to load 10 digits

```
function preload() {
    digit0 = loadImage('assets/0.png');
    digit1 = loadImage('assets/1.png');
    digit2 = loadImage('assets/2.png');
    digit3 = loadImage('assets/3.png');
    digit4 = loadImage('assets/4.png');
    digit5 = loadImage('assets/5.png');
    digit6 = loadImage('assets/6.png');
    digit7 = loadImage('assets/7.png');
    digit8 = loadImage('assets/8.png');
    digit9 = loadImage('assets/9.png');

    let prefix = 'assets/';
    let postfix = '.png';
    let filename;
    for (let count=0; count<10; count++) {
        filename = prefix + count + postfix;
        console.log(filename);
    }
}

```

Check JavaScript console in the Web Browser



Not efficient if I have to use 10 variables for 10 digits.

Step 3: Change to use a JS Array

```
// first lesson to use arrays
// go slow
let digitImgs = [];
let digit0;
```

Empty JS array
Use [ ]

```
let scoreNumber = 0;
let scoreSprite;

function preload() {
    // load 10 digits into a JS Array
    let prefix = 'assets/';
    let postfix = '.png';
    let filename;
    for (let count=0; count<10; count++) {
        filename = prefix + count + postfix;
        // console.log(filename);
        digitImgs[count] = loadImage(filename);
    }
}
```

Each image is loaded one by one into the array using the index number

```
function setup() {
    new Canvas(400,600);

    scoreSprite = new Sprite();
    scoreSprite.img = digitImgs[0];
    scoreSprite.collider = "none"; // explain static, dynamic, none
}

```

Use the first one in the array.

Step 4: Add key pressed to change the score value

```
function draw() {
    background("skyblue"); // erase the trail left behind

    if (kb.presses("up")) {
        scoreNumber++;
    }

    if (kb.presses("down")) {
        scoreNumber--;
    }

    // keep the variable within 0-9
    scoreNumber = constrain(scoreNumber, 0, 9);

    textSize(16);
    text("Score is " + scoreNumber, 50,50);
}

```

Checking for up arrow key pressed

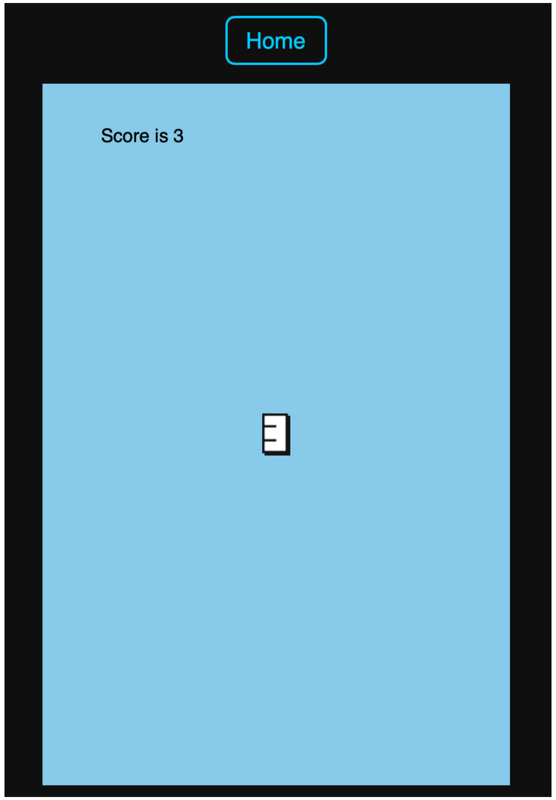
constrain() function can restrict the values of any variable.

This is for testing purposes.
We want to see it on the screen so that we can change our digits images too.

Step 5: as the variable value changes, change the digit images

```
// keep the variable within 0-9
scoreNumber = constrain(scoreNumber, 0, 9);
scoreSprite.img = digitImgs[scoreNumber];

```

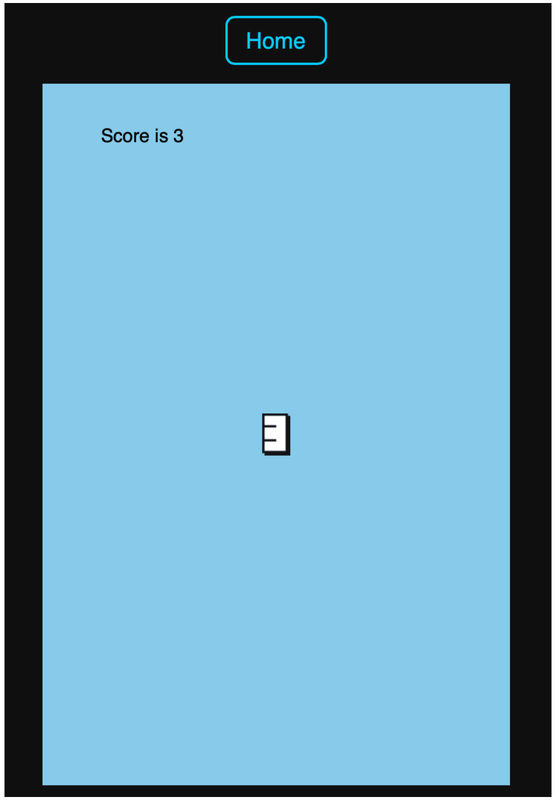
Set the correct digit image



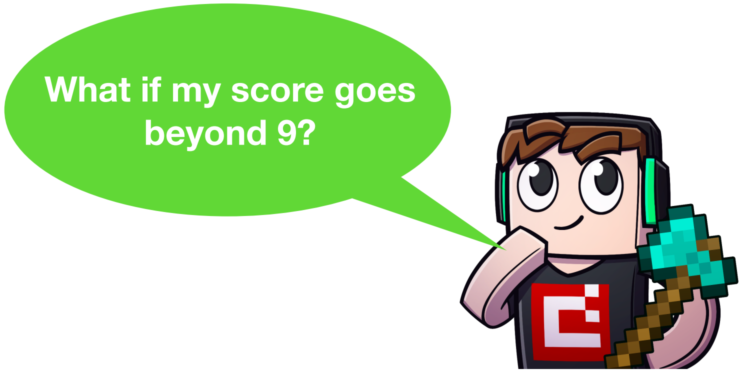
Press up or down arrow keys
Observe how the score variable
and digits images are updating too.

Step 5: as the variable value changes, change the digit images

```
    // keep the variable within 0-9
    scoreNumber = constrain(scoreNumber, 0, 9);
    scoreSprite.img = digitImgs[scoreNumber];

```



**Press up or down arrow keys
Observe how the score variable
and digits images are updating too.**



Step 6: Handle 2 digits or even 3 digits score (part 1)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

Index position: 0 1

p5 JS library has convenient methods

p5.js

Reference
Tutorials
Examples
Contribute
Community
About

English

Accessibility

Reference > constrain()

constrain()

Constrains a number between a minimum and maximum value.

p5.js

Reference
Tutorials
Examples
Contribute
Community
About

Start Coding

Donate

Reference

English

Accessibility

Reference > str()

str()

Converts a `Boolean` or `Number` to `String`.

`str()` converts values to strings. See the String reference page for guidance on using template literals instead.

The parameter, `n`, is the value to convert. If `n` is a Boolean, as in `str(false)` or `str(true)`, then the value will be returned as a string, as in `'false'` or `'true'`. If `n` is a number, as in `str(123)`, then its value will be returned as a string, as in `'123'`. If an array is passed, as in `str([12.34, 56.78])`, then an array of strings will be returned.

Step 6: Handle 2 digits or even 3 digits score (part 2)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

```
    // keep the variable within 0-9
    scoreNumber = constrain(scoreNumber, 0, 999);

    // call custom function
    drawScore();
```

New custom function
To make it easy to merge codes later into flappybird.js file.

Index position: 0 1

```
// use function to group codes within a context together
function drawScore() {
    // scoreSprite.img = digitImgs[scoreNumber];
    let scoreString = str(scoreNumber);
    // console.log("score as a string: " + scoreString);
    let scoreDigitArray = scoreString.split("");

    text("Score array size is " + scoreDigitArray.length, 50,70);
    text("Score array at index 1 is " + scoreDigitArray[0], 50,90);

}

```

Debugging the variables

JavaScript has convenient methods too

In JavaScript, several methods can be used to convert a string into an array, depending on the desired outcome: Using split().

This is the most common and versatile method for splitting a string into an array of substrings based on a specified `separator`.

JavaScript

```
const myString = "apple,banana,orange";
const myArray = myString.split(","); // Splits by comma
console.log(myArray); // Output: ["apple", "banana", "orange"]

const anotherString = "hello world";
const charArray = anotherString.split(""); // Splits by each characte
console.log(charArray); // Output: ["h", "e", "l", "l", "o", " ", "w"
```

Step 6: Handle 2 digits or even 3 digits score (part 3)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

Index position: 0 1

```
    // keep the variable within 0-9
    scoreNumber = constrain(scoreNumber, 0, 999);

    // call custom function
    drawScore();
```

```
// use function to group codes within a context together
function drawScore() {
    // scoreSprite.img = digitImgs[scoreNumber];
    let scoreString = str(scoreNumber);
    // console.log("score as a string: " + scoreString);
    let scoreDigitArray = scoreString.split("");


    text("Score array size is " + scoreDigitArray.length, 50,70);
    text("Score array at index 1 is " + scoreDigitArray[0], 50,90);

}

```

Split the String into an Array

Step 6: Handle 2 digits or even 3 digits score (part 4)

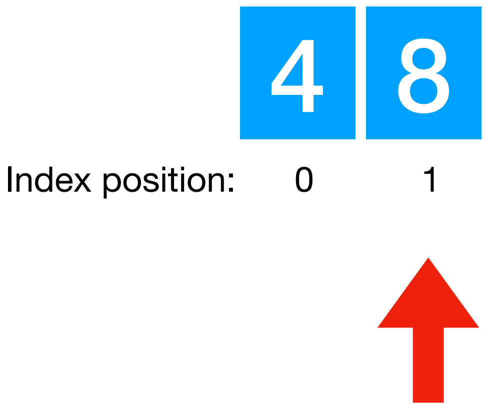
- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

Index position: 0 1

How do you get the digit image for index position 0?

Step 6: Handle 2 digits or even 3 digits score (part 5)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array



How do you get the digit image for index position 1?

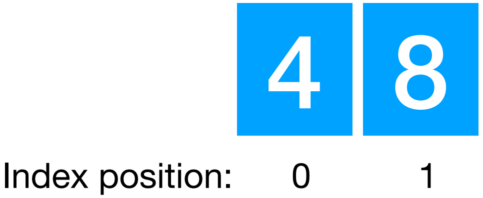
Step 6: Handle 2 digits or even 3 digits score (part 6)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

	4	8
Index position:	0	1

Step 6: Handle 2 digits or even 3 digits score (part 7)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array



Step 6: Handle 2 digits or even 3 digits score (part 8)

- Score variable as a number, is easier for counting points in a game
- But for displaying, we can convert it to a String for ease of taking out each digit
- From String, its easy to break it up into JS Array

Index position: 0 1

Step 7: Use Sprite Group (part 1)

Index position: 0 1

```
let digitImgs = [];

let scoreNumber = 11;
let scoreSprite;
let scoreGroup;

```

Use Sprite Group

```
function setup() {
    new Canvas(400,600);

    // no need
    // scoreSprite = new Sprite();
    // scoreSprite.img = digitImgs[0];
    // scoreSprite.collider = "none"; // explain static, dynamic, none

    // use Sprite Group to manage the digits
    scoreGroup = new Group();
    scoreGroup.collider = "none";
    scoreGroup.layer = 1000;
}

```

Use Sprite Group

Step 7: Use Sprite Group (Part 2)

```
// use function to group codes within a context together
function drawScore() {

    // clear previous score digits sprites
    scoreGroup.removeAll();

    // number to a String value
    let scoreString = str(scoreNumber);
    // String to an Array
    let scoreDigitArray = scoreString.split("");

    let offset = 0;
    let middle = width/2; // center of screen

    for (let one of scoreDigitArray) {
        let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
        oneSprite.img = digitImgs[one];
        offset = offset + 25; // move right by size of image width + 1px
    }
}

```

Remove all members

Add member to the Group

X position

Y position

```
74.let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
```

???

The sprite is added as a member of the Sprite Group

X position

Y position

```
74.let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
```

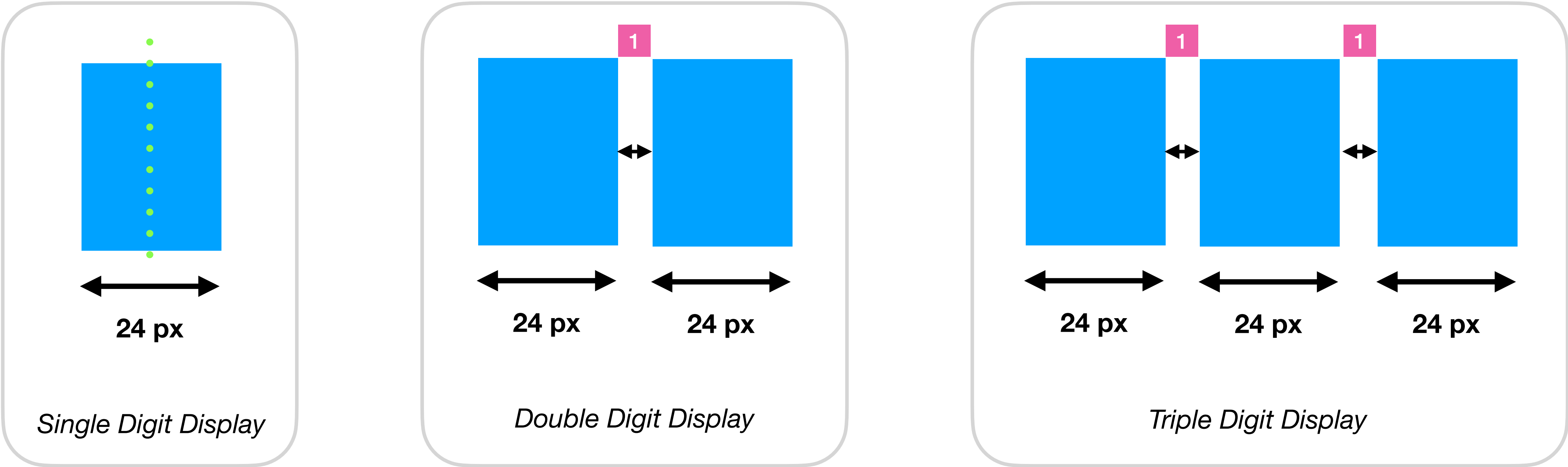
The sprite is added as a member of the Sprite Group

???

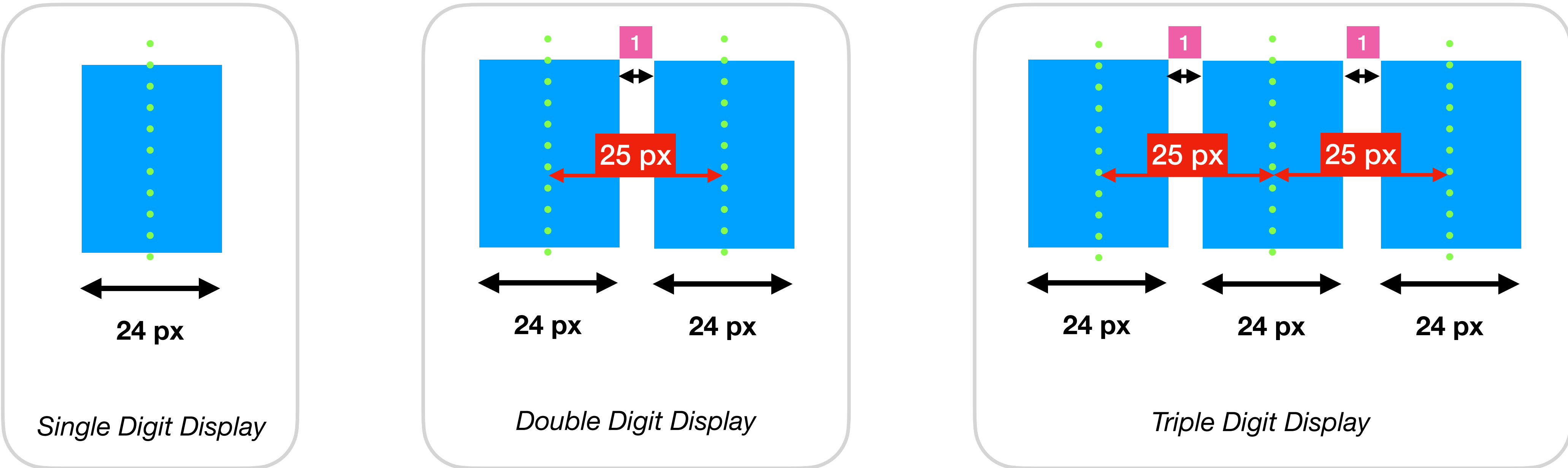
```
    let offset = 0;
    let middle = width/2; // center of screen

    for (let one of scoreDigitArray) {
        let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
        oneSprite.img = digitImgs[one];
        offset = offset + 25; // move right by size of image width + 1px
    }

```

offset is used to add spacing between the digits

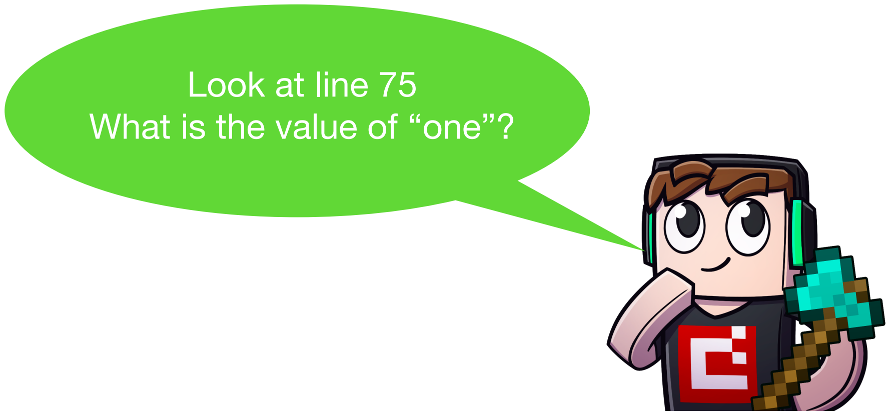
1 **Leave a gap of 1 px between the digits**



Considering the difference of 25 px, we can know each sprite’s xpos (centre point)



```
70.     let offset = 0;
71.     let middle = width/2; // center of screen

73.     for (let one of scoreDigitArray) {
74.         let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
75.         oneSprite.img = digitImgs[one];
76.         offset = offset + 25; // move right by size of image width + 1px
77.     }
78.
```



Task: Add console.log() to print the variable "one".

Step 7: Use Sprite Group (Part 2)

```
// use function to group codes within a context together
function drawScore() {

    // clear previous score digits sprites
    scoreGroup.removeAll();

    // number to a String value
    let scoreString = str(scoreNumber);
    // String to an Array
    let scoreDigitArray = scoreString.split("");

    let offset = 0;
    let middle = width/2; // center of screen

    for (let one of scoreDigitArray) {
        let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
        oneSprite.img = digitImgs[one];
        offset = offset + 25; // move right by size of image width + 1px
    }
}

```

Remove all members

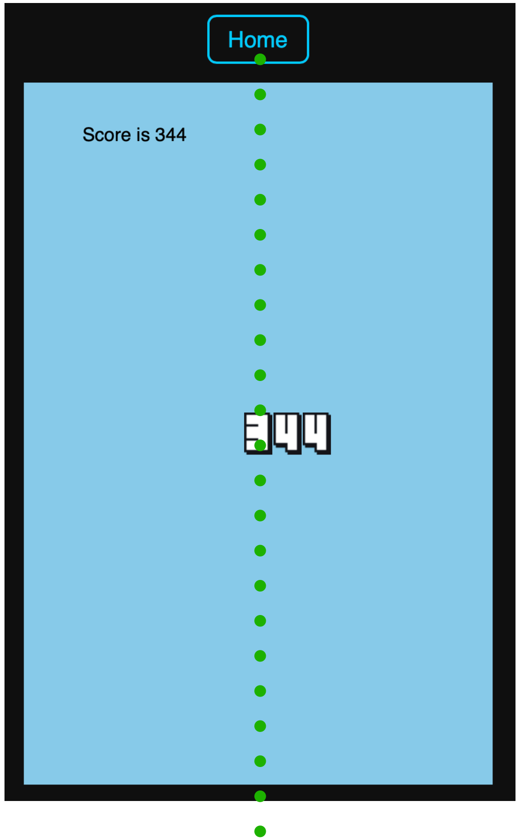
Add member to the Group

WHAT IF your score to a 3 digits number???

Observe: The digits are not in the centre of the screen.

```
    if (kb.presses("3")) {
        scoreNumber = 345;
    }
    // keep the variable within 0-999
    scoreNumber = constrain(scoreNumber, 0, 999);

```

A quick way to increase the score to 3 digits value.

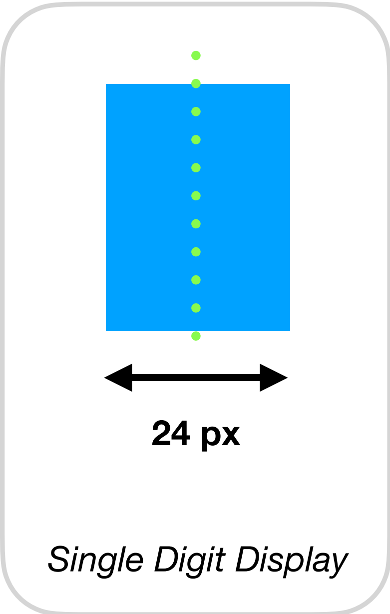


Display Score in the Center

Explanation

Display Score in the Center of the Game

Easy. Just follow camera.x

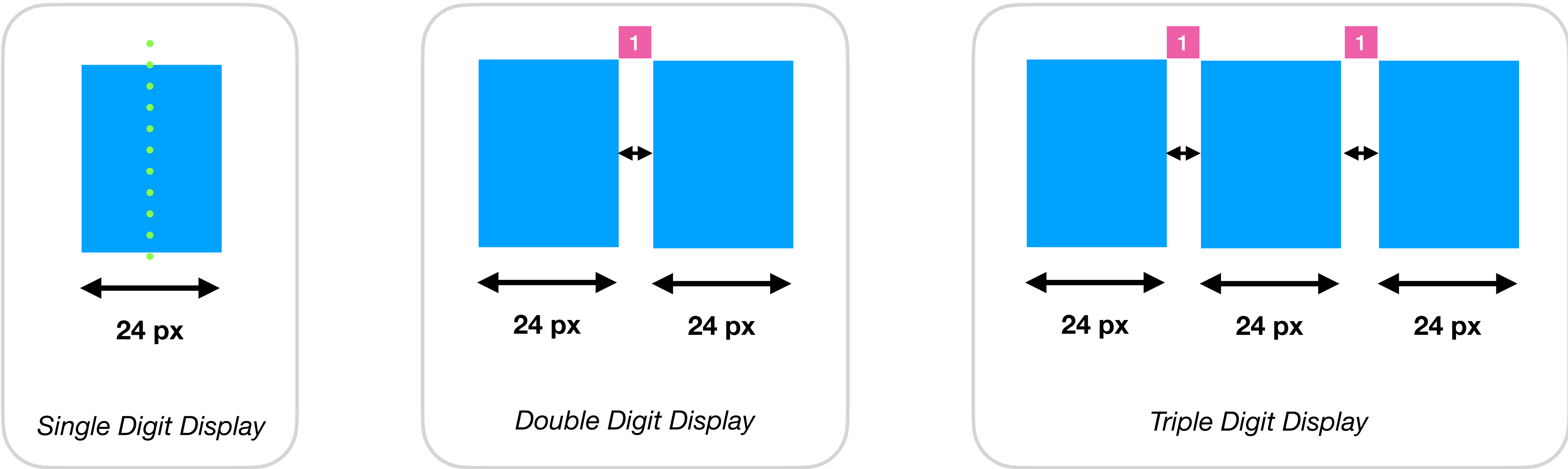


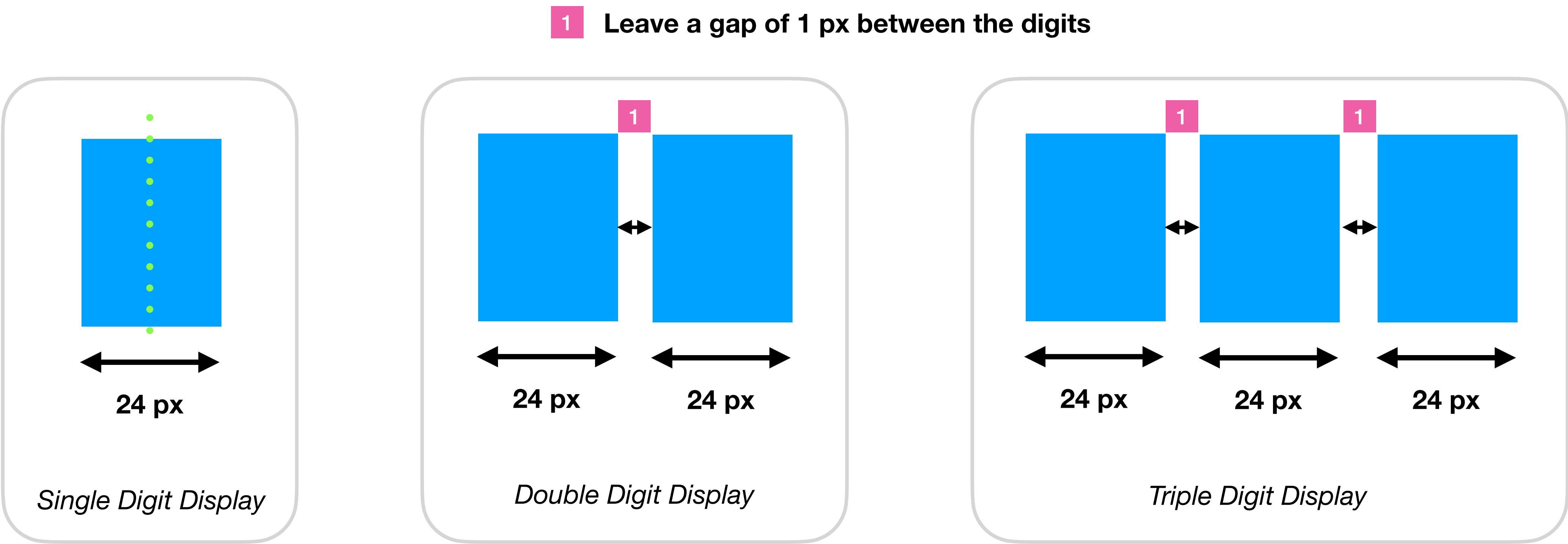
Digit Image width: 24px

new Sprite(xpos, ypos)

*xpos and ypos is in
the centre of the
given width and height.

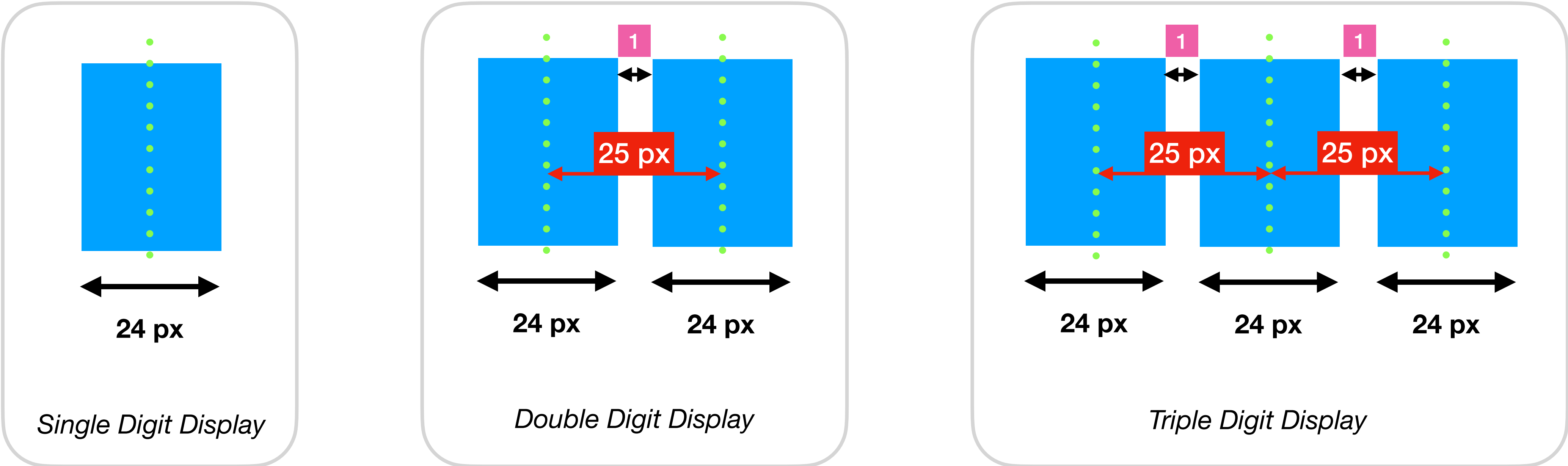
1 **Leave a gap of 1 px between the digits**



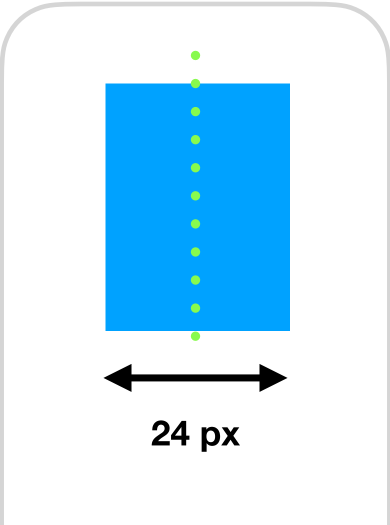


1. We can use Sprite Group, to manage all the digits in the Score value.

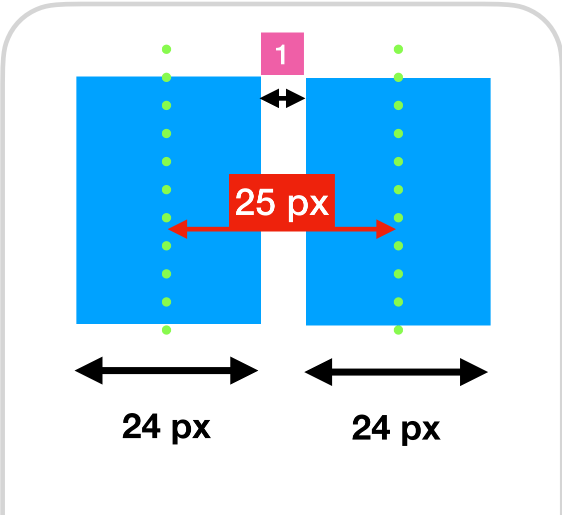
2. Considering the difference of 25 px, we can know each sprite's xpos (centre point)



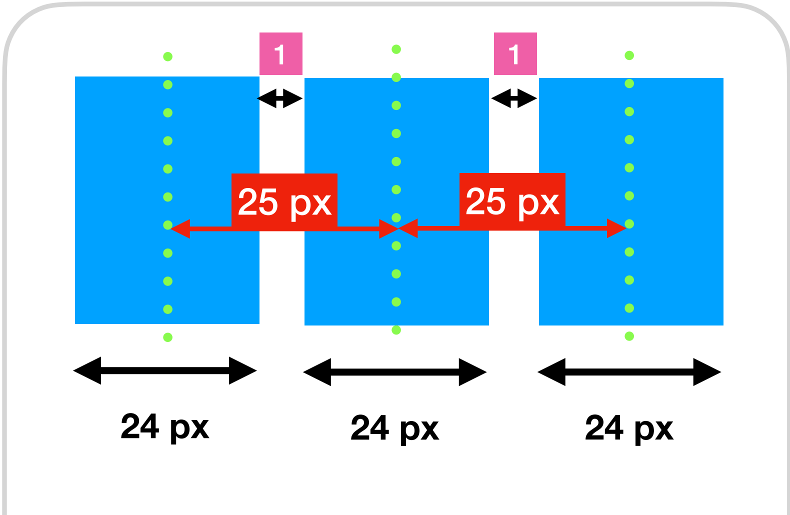
2. Considering the difference of 25 px, we can know each sprite’s xpos (centre point)



Single Digit Display

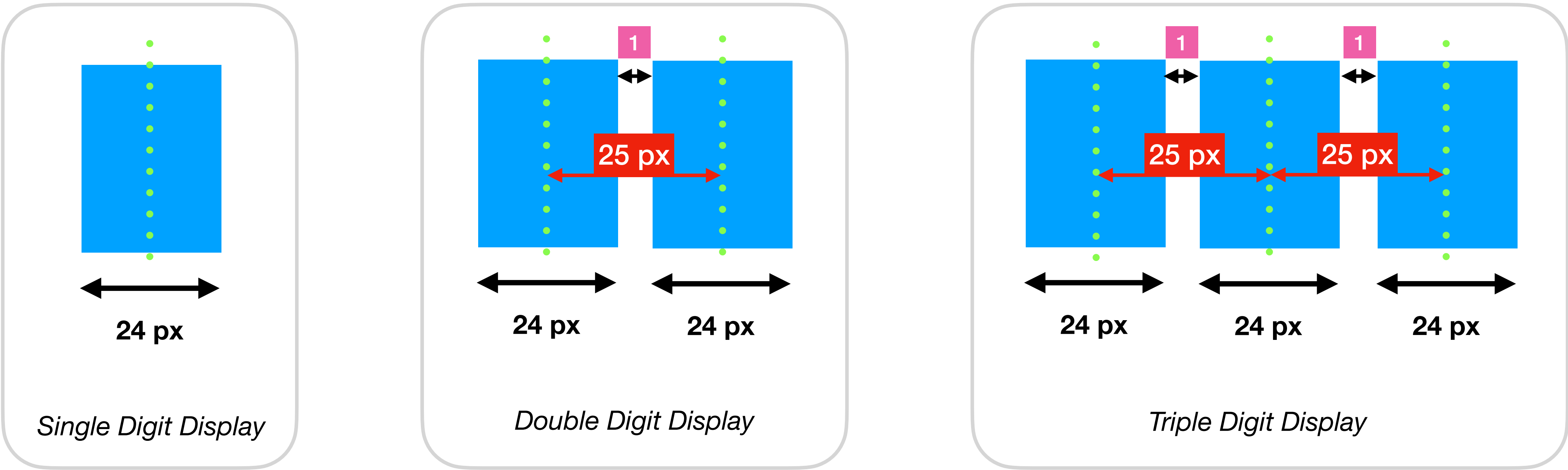


Double Digit Display



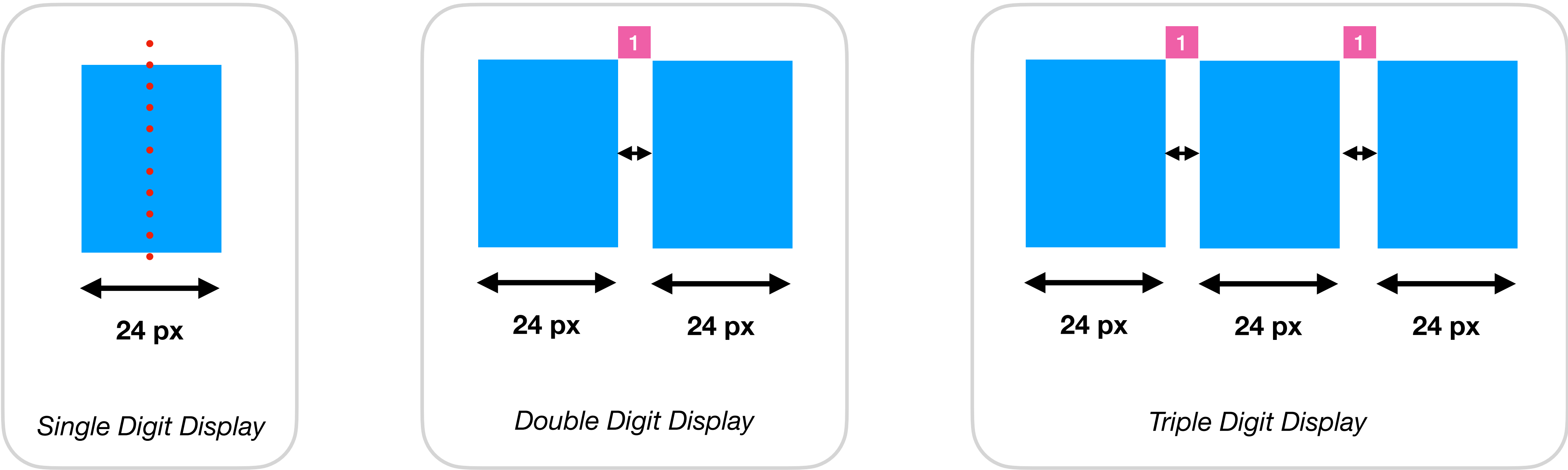
Triple Digit Display

```
The pattern is…

1st Digit:  0*25
2nd Digit:  1*25
3rd Digit:  2*25
```



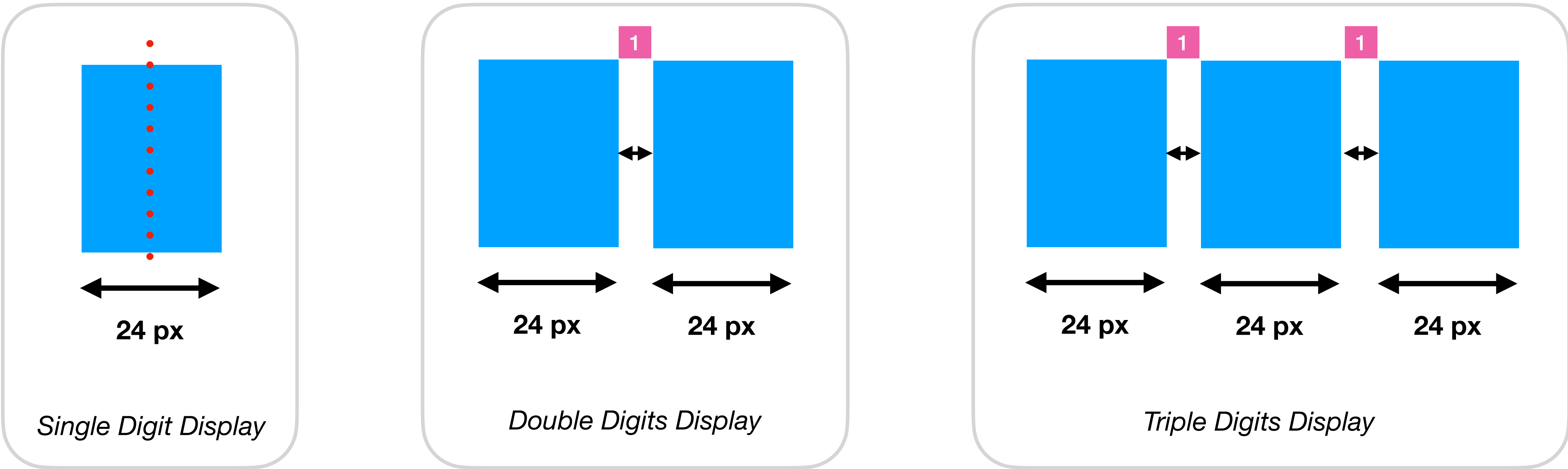
Now we know how to set xpos for each Sprite.

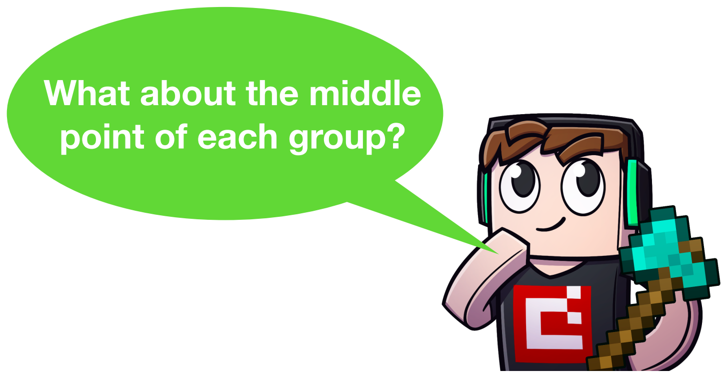
To find the middle value for 3 different test cases…



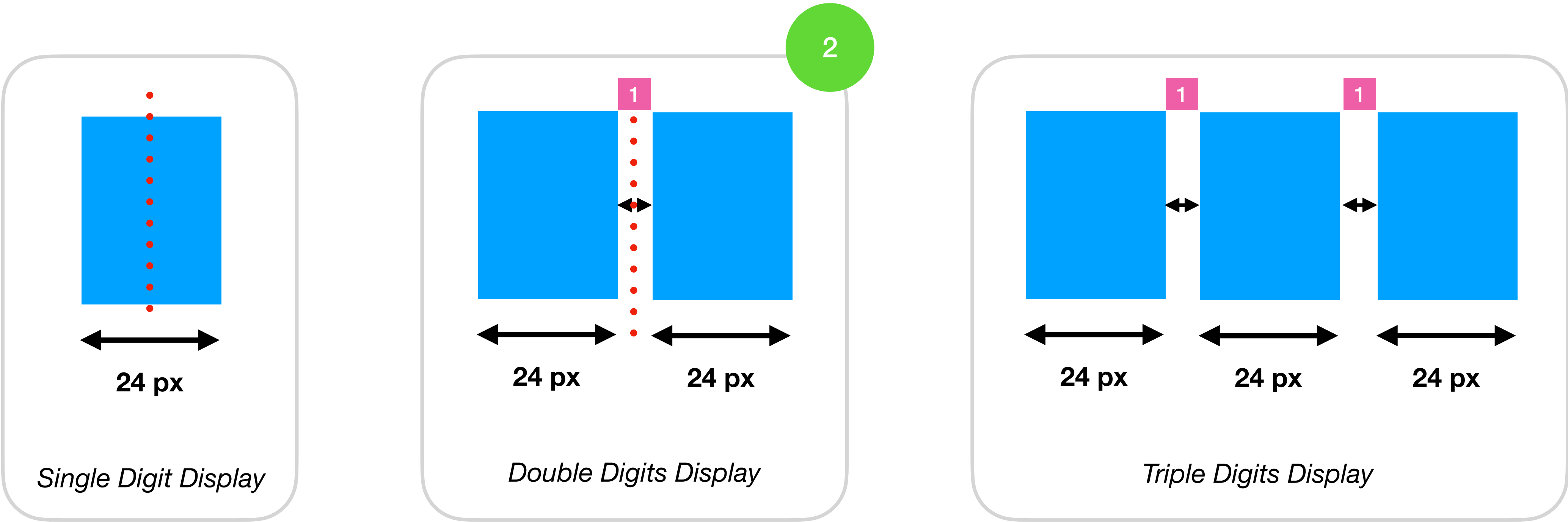
Single digit is always easy.

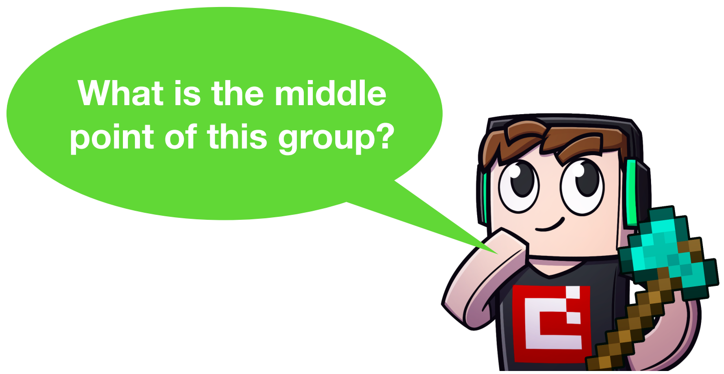
To find the middle value for 3 different test cases…



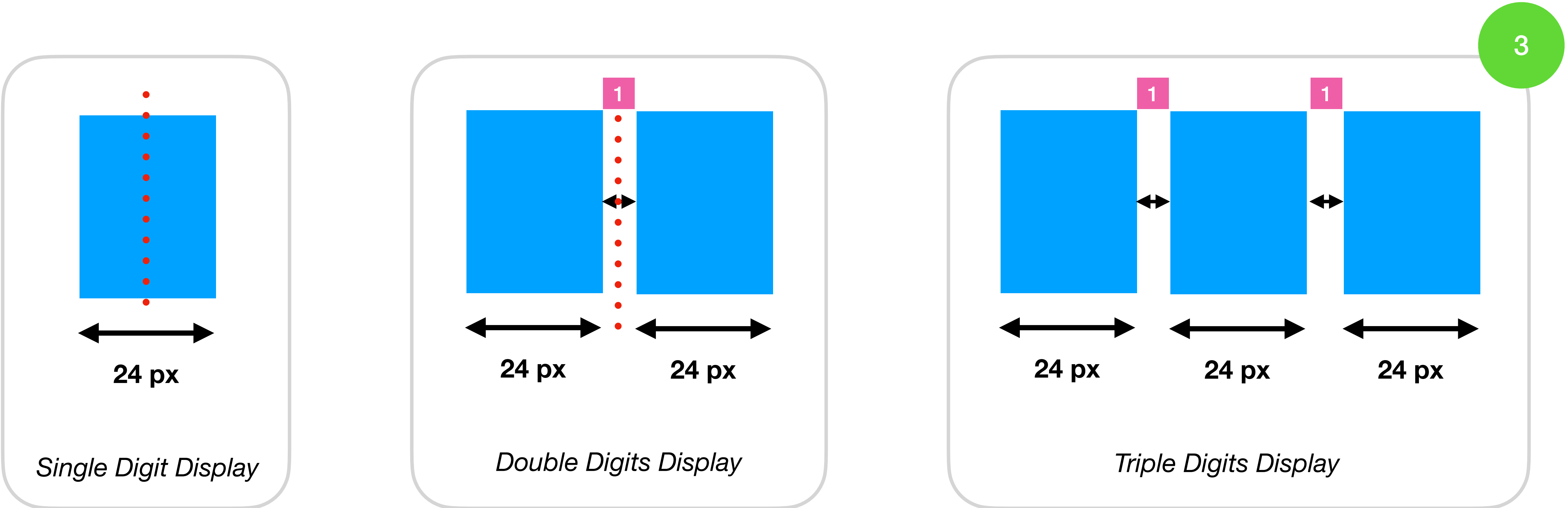


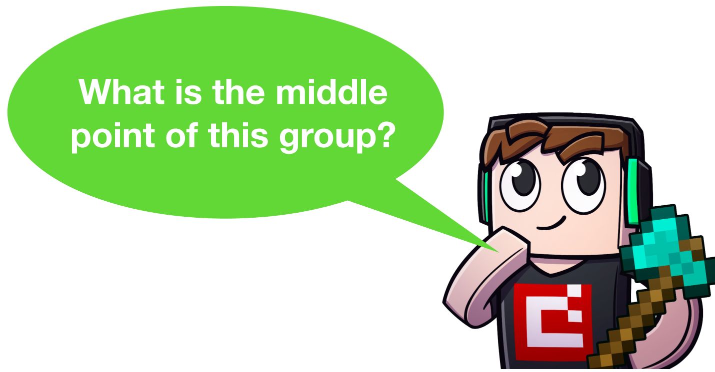
To find the middle value for 3 different test cases…



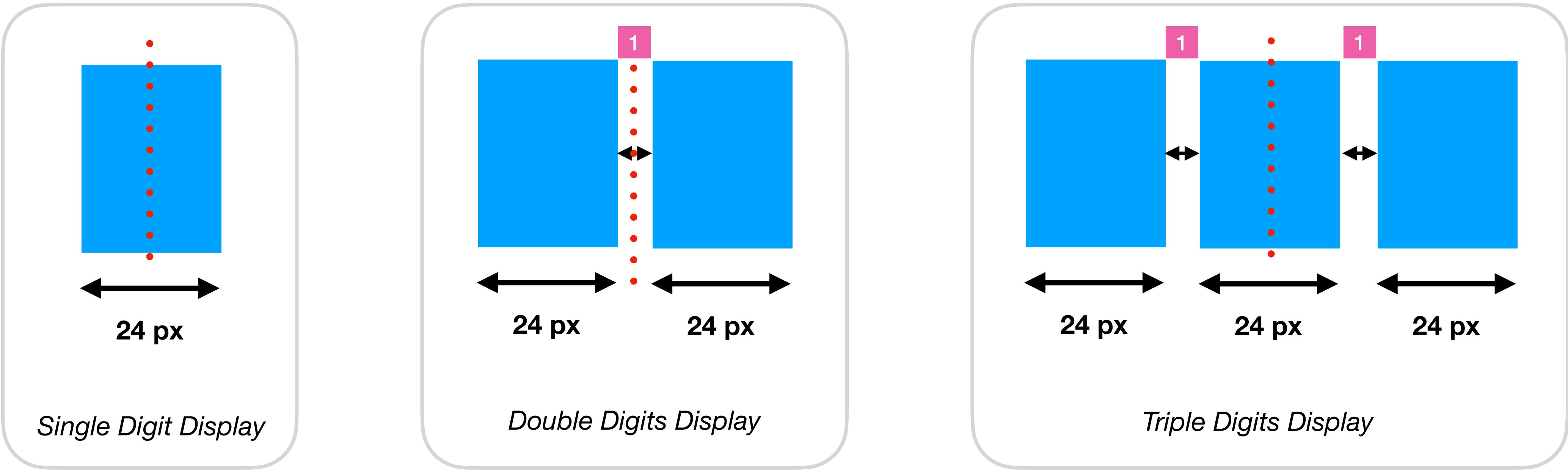


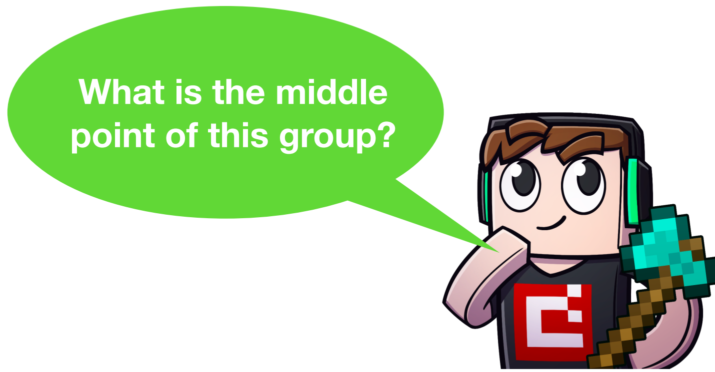
To find the middle value for 3 different test cases…



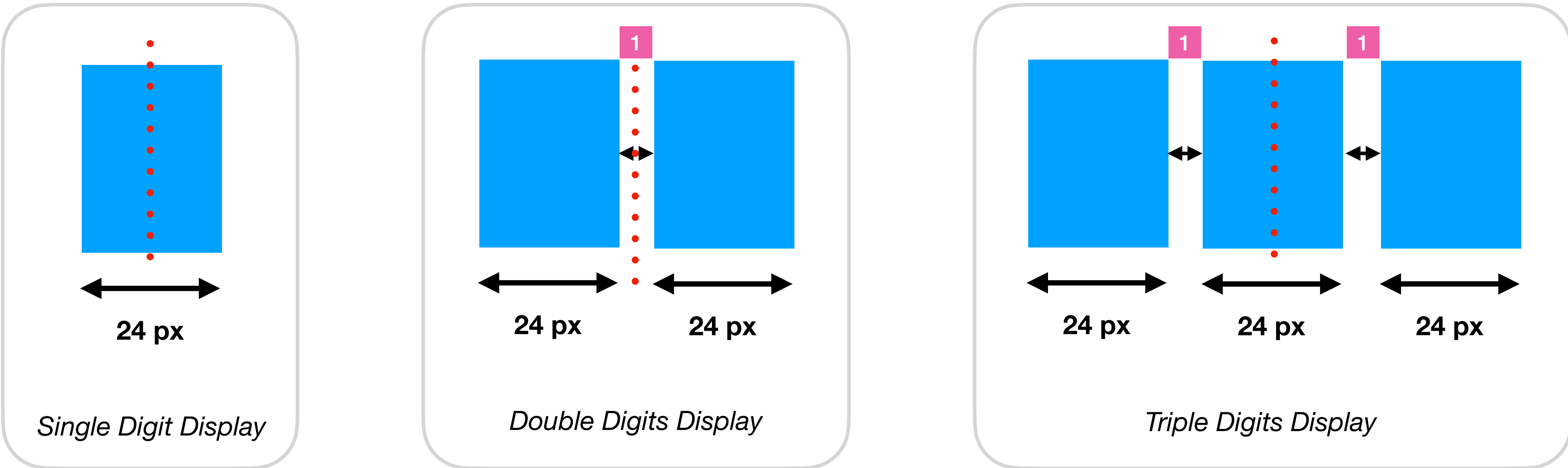


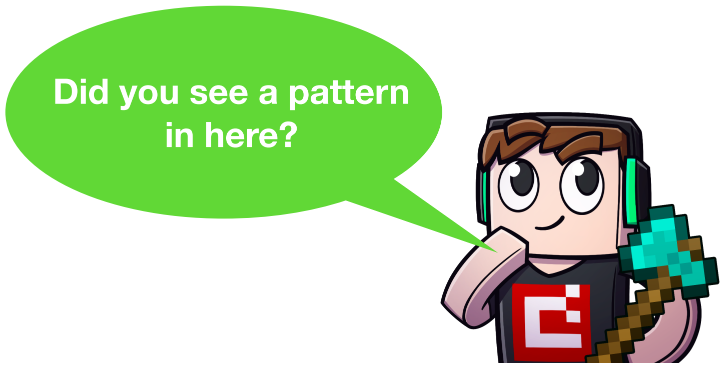
To find the middle value for 3 different test cases…

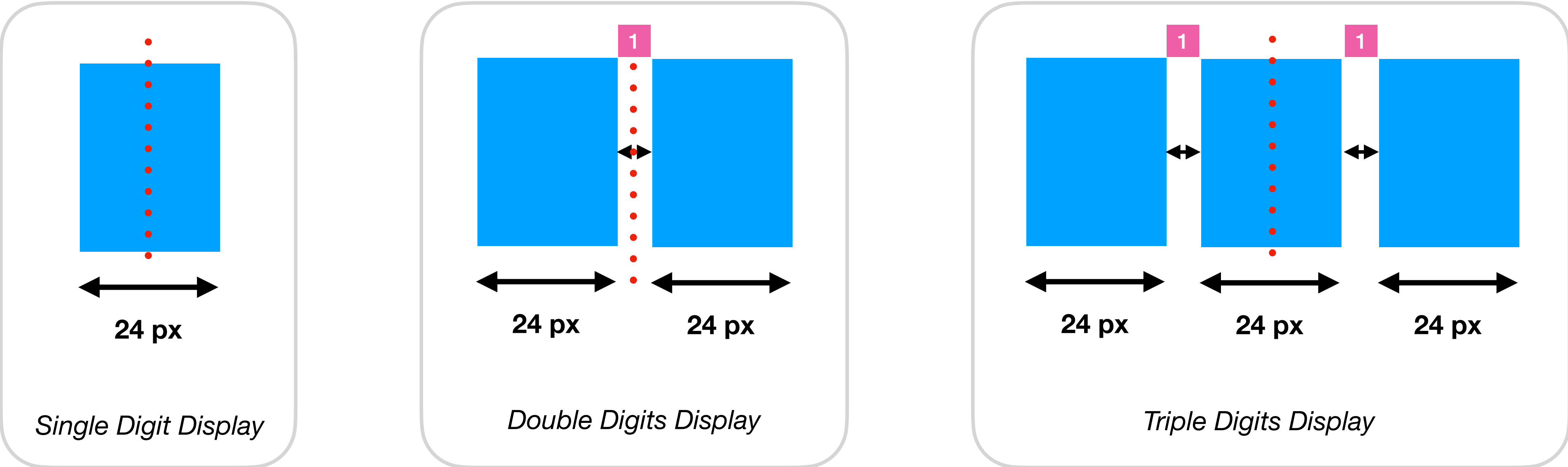


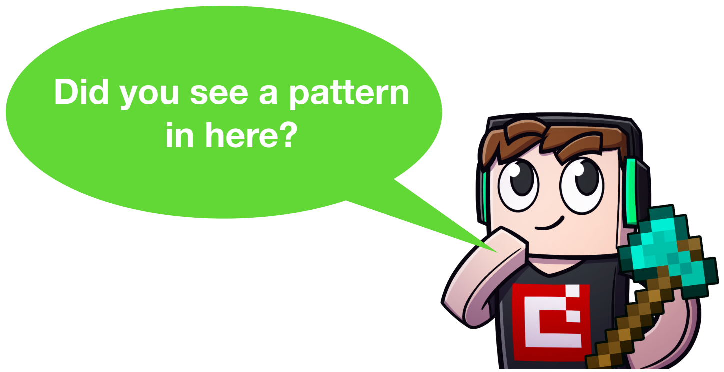


To find the middle value for 3 different test cases…

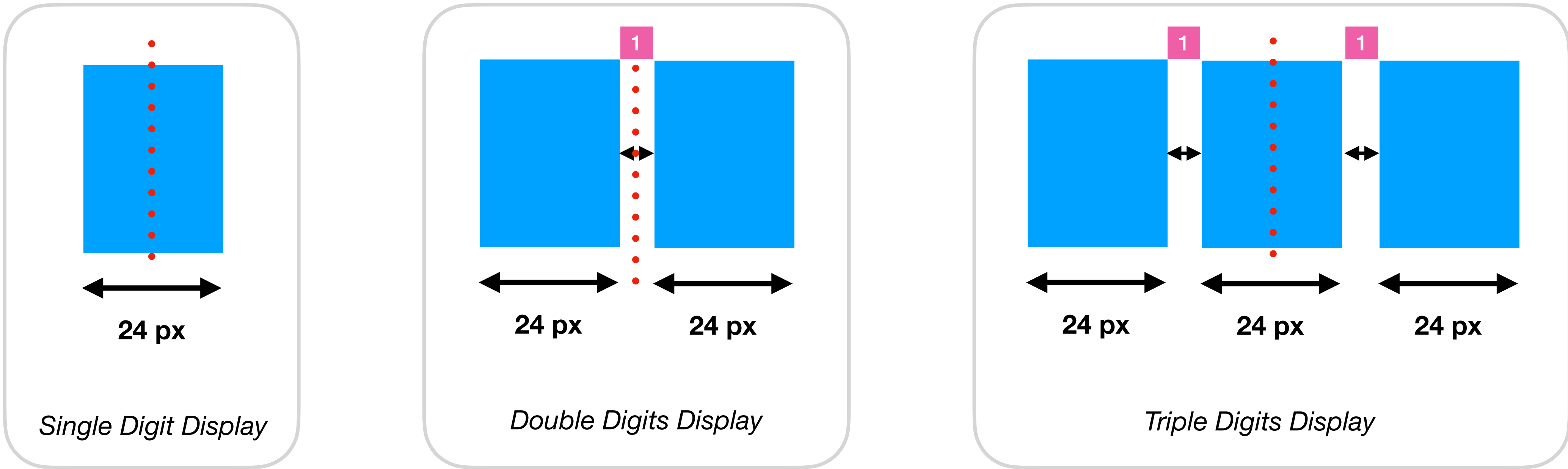






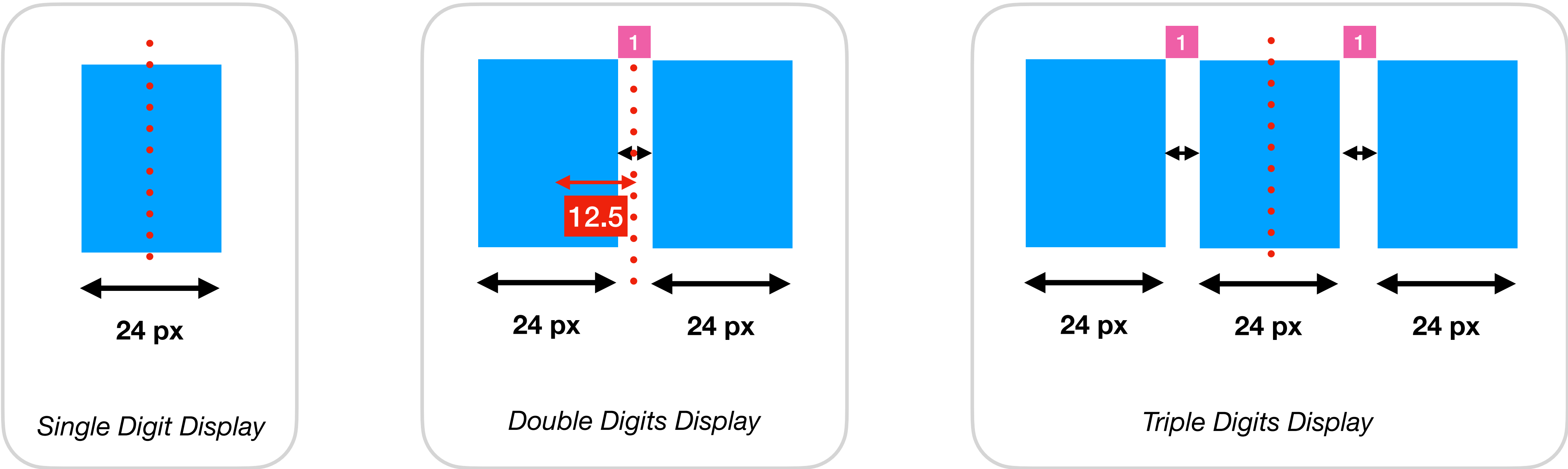


Find the total width of everything and divide by 2

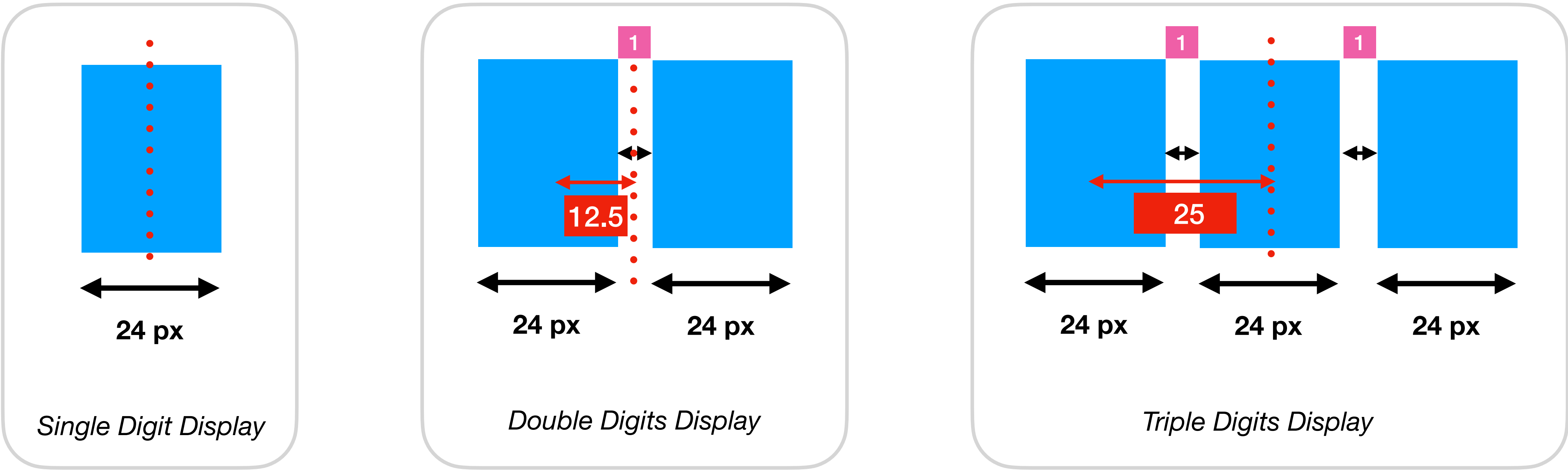


Find the total width of everything and divide by 2

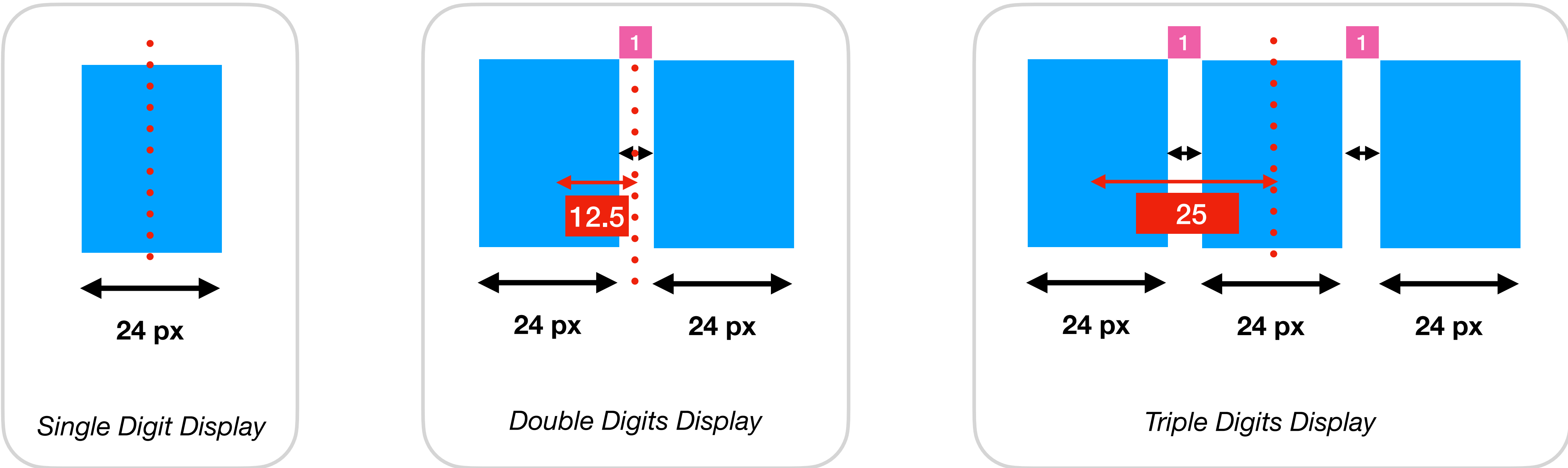
Move each sprite's X position one by one…

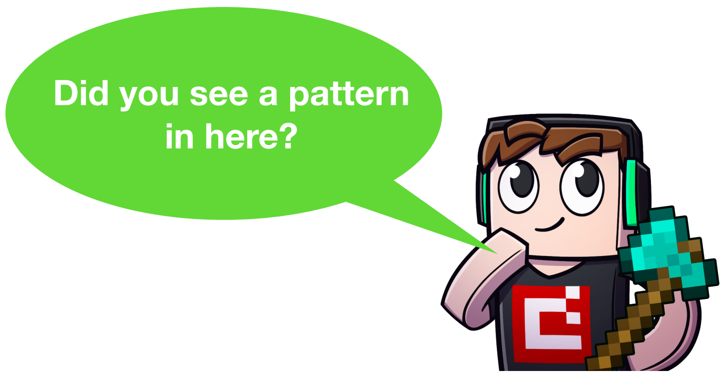


Move each sprite's X position one by one…

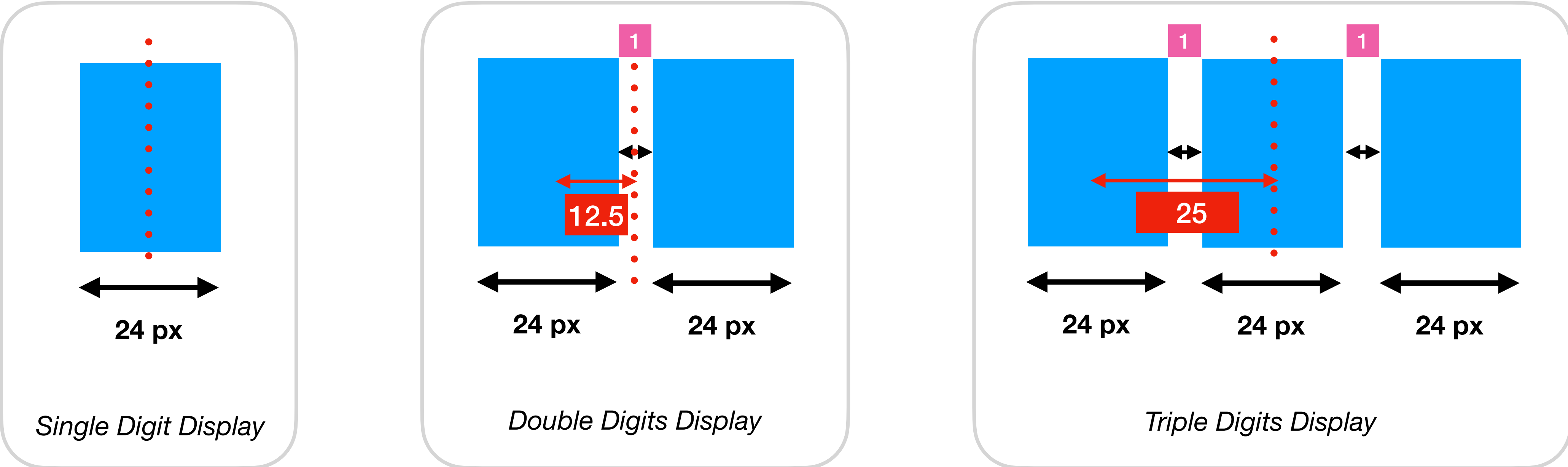


Move each sprite's X position one by one…

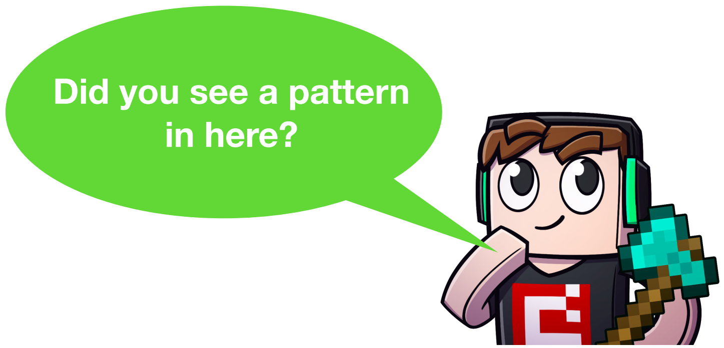




Move each sprite’s X position one by one…



(Number of digits - 1) *25 /2



PREVIOUSLY we have...

Step 7: Use Sprite Group (Part 2)

```
// use function to group codes within a context together
function drawScore() {

    // clear previous score digits sprites
    scoreGroup.removeAll();

    // number to a String value
    let scoreString = str(scoreNumber);
    // String to an Array
    let scoreDigitArray = scoreString.split("");

    let offset = 0;
    let middle = width/2; // center of screen

    for (let one of scoreDigitArray) {
        let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
        oneSprite.img = digitImgs[one];
        offset = offset + 25; // move right by size of image width + 1px
    }
}

```

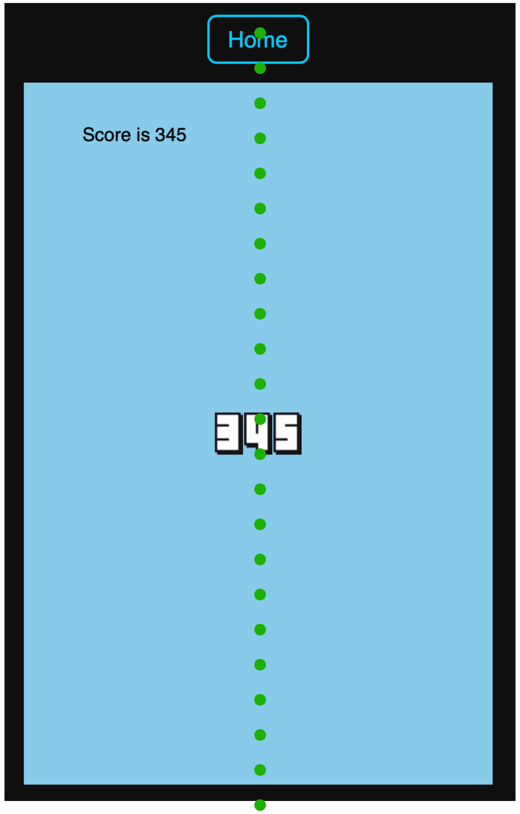
Remove all members

Add member to the Group

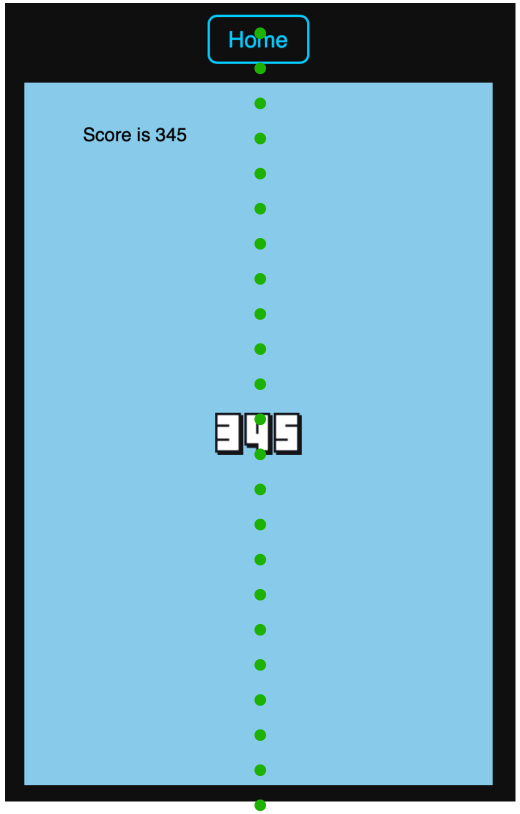
Now the codes looking like this…

```
for (let one of scoreDigitArray) {
        let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
        // move to center
        oneSprite.x = oneSprite.x - (scoreDigitArray.length-1)*25/2;

        oneSprite.img = digitImgs[one];
        offset = offset + 25; // move right by size of image width + 1px
    }

```

Subtract using the formula



Observe: The digits are in the centre of the screen.



Merging Codes

Merging Codes

1. Look at the variables section, copy over

2. Look at the preload() function, copy over

3. Look at the setup() function, for the sprite group for score. Copy over

4. Copy the drawScore() function, after the spawnPipePair() function

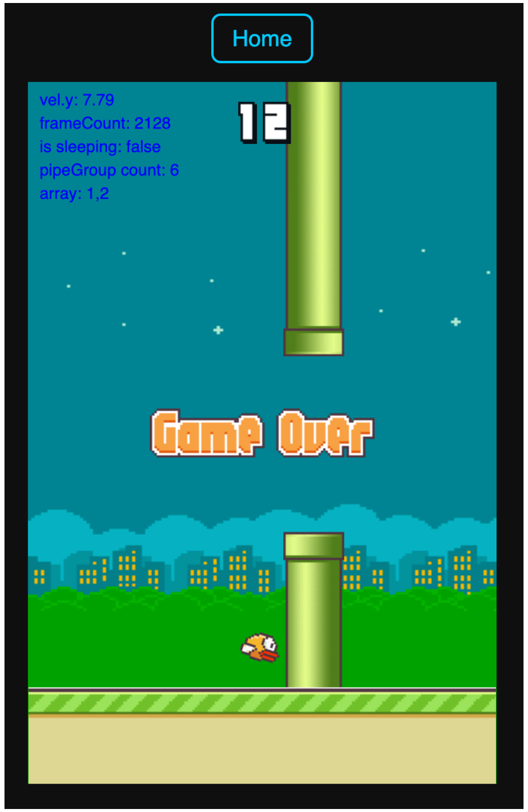
5. Look inside draw() function, decide on where do you want to add the line to call drawScore()

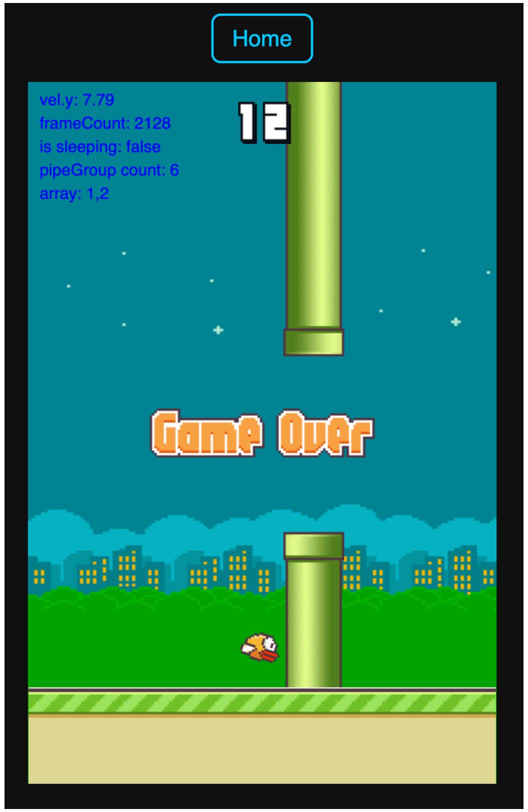
How to follow the camera?

Final Step

```
// let middle = width/2; // center of screen
let middle = camera.x; // follow the camera
```

- Find the middle variable inside drawScore() function
- Instead of width/2
- Just change it to middle = camera.x
- Easy





The Y position of the score doesn't look like this?

Where do you change the code?

```
let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
```

Display in the middle of the canvas.

```
let oneSprite = new scoreGroup.Sprite(middle+offset, height/2, 24, 36);
```

Display in the middle of the canvas.

```
let oneSprite = new scoreGroup.Sprite(middle+offset, 35, 24, 36);
```

Display somewhere on the top of the canvas.

Score is still 0…

19 Inside spawnPipePair() - Updating Scores (1)

Write the code to update the scores.

- We have a global score variable set to 0.
- Detect if the bird have passed the pipe to increase the score.

```
function spawnPipePair(){
  // < . . . previous code . . . >

  // create the top pipe <. . . previous code . . .>
  // topPipe = new Sprite(bird.x + 400, midY - gap / 2 - 200, 52, 320, 'static');
  // topPipe.img = pipe;
  // topPipe.rotation = 180;

  // Add to one pipe per pair (top or bottom)
  topPipe.passed = false; // Add this property to topPipe

  // < . . . previous code . . . >
}
```

We will set topPipe.passed to true once the bird has passed this particular pipe.

20 Inside draw() - Updating Scores (2)


```
function draw() {
  // < . . . previous code . . . >

  if (startGame){
    // < . . . previous code inside startGame condition. . . >
    // < . . reference position . . .>
    // for (let pipe of pipeGroup){
    //   if (pipe.x < -50){
    //     pipe.remove();
    //   }
    // }

    // increase score if pipe passed
    for (let pipe of pipeGroup) {
      // center pos + half pipe width = right edge pos
      let pipeRightEdge = pipe.x + pipe.w / 2;

      // center pos - half bird width = left edge pos
      let birdLeftEdge = bird.x - bird.w / 2;

      // compare x-coordinates of player and pipes
      if (pipe.passed == false && pipeRightEdge < birdLeftEdge){
        pipe.passed = true;
        score++;
      }
    }
  }
}
```

1. *This code must be in the* ***startGame*** *condition that you defined earlier.*
2. *Recall that startGame is inside the* ***draw()*** *function*
3. *The for loop will* ***loop through every set of pipes***
   a) *If this* ***current pipe has gone to the left of the bird***
   b) *It means that the bird successfully flew past it.*
      i. *Set the* ***passed property to true***
      ii. *Increase the* ***score by 1***

Other observations

- Game Over Message sometimes is rotated…
- Look at the sprite Label, what is the collider? If you don't set anything, its "dynamic" - meaning it will collide with the bird
- Set collider to "none".

Game restart after a short delay…

JavaScript

The `setTimeout()` method in JavaScript schedules the execution of a function or a piece of code after a specified delay. It is a fundamental tool for managing asynchronous operations and creating time-based interactions in web applications and Node.js environments.

Syntax:

JavaScript

```javascript
let timerId = setTimeout(function, delay, arg1, arg2, ...);
```

Parameters:

- `function`: The function to be executed after the delay. This can be a named function or an anonymous function.
- `delay`: The time in milliseconds (1000 milliseconds = 1 second) to wait before executing the function.
- `arg1, arg2, ...` (Optional): Additional arguments to be passed to the function when it is executed.

How it works:

- `setTimeout()` registers the provided function to be executed after the specified `delay`.
- It returns a `timerId`, which is a unique identifier for the created timer.
- The main thread of execution continues to run subsequent lines of code immediately after `setTimeout()` is called, without waiting for the scheduled function to execute. This is what makes `setTimeout()` an asynchronous operation.
- Once the `delay` expires, the registered function is added to the event queue and will be executed when the call stack is clear.

```javascript
// You can also use an anonymous function:
setTimeout(() => {
  console.log("This message appears after 1 second.");
}, 1000);
```

Other observations

- After game restart, I can see the old score is still there.
- Add one more line after setting score = 0
- scoreGroup.removeAll() so that no score is displaying

Challenge

How to switch between day and night?